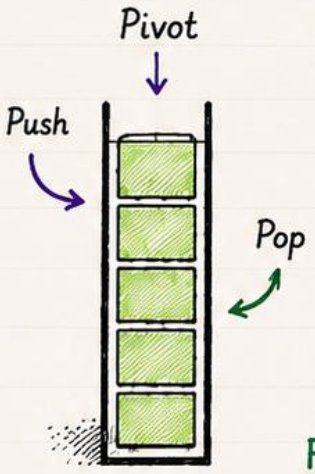
Date : __/__/____

MASTER CODE NOTES

From Beginners to Advanced

QUICK SORT



- Code.
- Optimize.
- Run.
- Debug.
- Repeat.

COMPLETE GUIDE

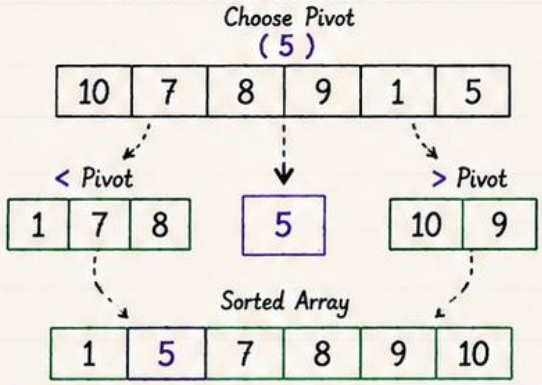
FROM BASICS TO INTERVIEW LEVEL

TOPIC 10 OF 10

WHAT'S INSIDE?

- Divide & Conquer Approach
- Partitioning & Pivot Selection
- Lomuto & Hoare Partition Schemes
- Quick Sort Algorithm & Dry Run
- Recursive Implementation (C, C++, Python)
- Time & Space Complexity
- Best, Average & Worst Cases
- Optimizations & Common Problems
- Interview Questions + Cheat Sheet
- Coding Templates & Examples

QUICK SORT IN ACTION



"Good Pivot + Good Partition + Smart Optimizations = Fast & Efficient Code!"

Think • Code • Solve • Succeed

codedsoul_05

Date : __/__/__

Divide
Conquer
Conquer

QUICK SORT

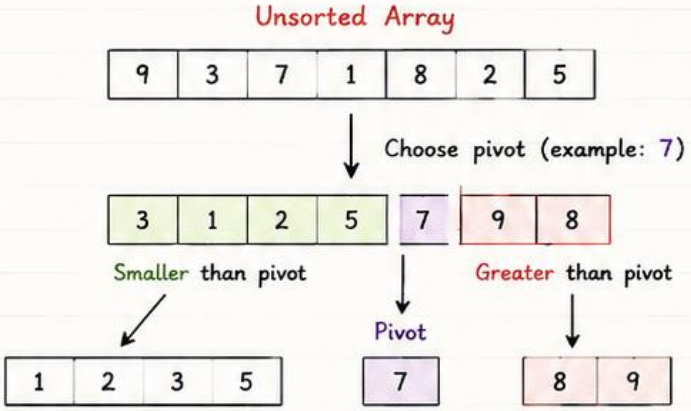
1. INTRODUCTION + BASIC CONCEPT

Quick Sort is a highly efficient, comparison-based, Divide and Conquer sorting algorithm.

It works by selecting a 'pivot' element and partitioning the array such that elements smaller than pivot are placed on the left and elements greater than pivot are placed on the right.

BASIC IDEA

1. Choose a pivot element.
2. Partition the array around the pivot.
3. Recursively apply the above steps to the left and right subarrays.



KEY POINTS

- It is an in-place sorting algorithm.
- Uses Divide and Conquer approach.
- Average Time Complexity is $O(n \log n)$.
- Worst Case Time Complexity is $O(n^2)$.
- Space Complexity is $O(\log n)$ on average (due to recursion stack).
- It is not a stable sorting algorithm.

EXAMPLE

Sort the array: [9, 3, 7, 1, 8, 2, 5]

Step 1: Choose pivot = 7

Step 2: Partition

=> Left: [3, 1, 2, 5] Pivot: [7] Right: [9, 8]

Step 3: Recursively sort left and right

=> Left sorted: [1, 2, 3, 5]

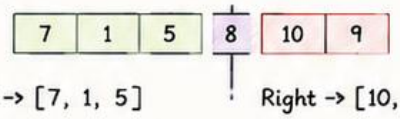
Right sorted: [8, 9]

Final Sorted Array: [1, 2, 3, 5, 7, 8, 9]

DRY RUN (SHORT)

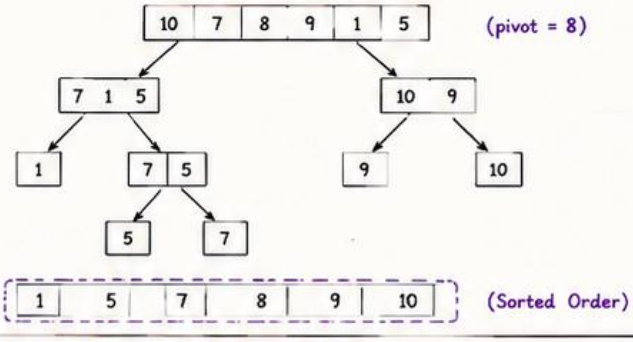
Array: [10, 7, 8, 9, 1, 5] Pivot = 8

Partition:



Left -> [7, 1, 5] Right -> [10, 9]

VISUAL RECURSION TREE



PROS

- Very fast in practice.
- In-place (no extra array needed).
- Cache friendly.

CONS

- Worst case $O(n^2)$.
- Performance depends on pivot choice.
- Not stable.

WHERE USED?

- Widely used in programming libraries.
- Efficient for large datasets.
- Basis of many advanced algorithms.

Remember: Good pivot choice = Good Performance!

Date : __/__/__

Divide
Conquer
Combine

2. DIVIDE & CONQUER APPROACH

Quick Sort follows the Divide and Conquer paradigm.
It breaks the problem into smaller subproblems, solves them recursively, and combines the results.

WHAT IS DIVIDE & CONQUER?

Divide and Conquer is an algorithm design technique that solves a problem by:

1. Divide: Break the problem into smaller subproblems of the same type.
2. Conquer: Recursively solve the smaller subproblems.
3. Combine: (If needed) Merge or combine the results to get the final solution.

HOW QUICK SORT FITS IN?

- The array is divided around a pivot.
- Elements smaller than pivot go to the left part.
- Elements greater than pivot go to the right part.
- These two parts are independently sorted using the same process.
- Finally, the whole array becomes sorted.

DIVIDE & CONQUER IN QUICK SORT

1. DIVIDE
Choose a pivot and partition the array into two parts:
Left (< pivot)
and Right (> pivot)

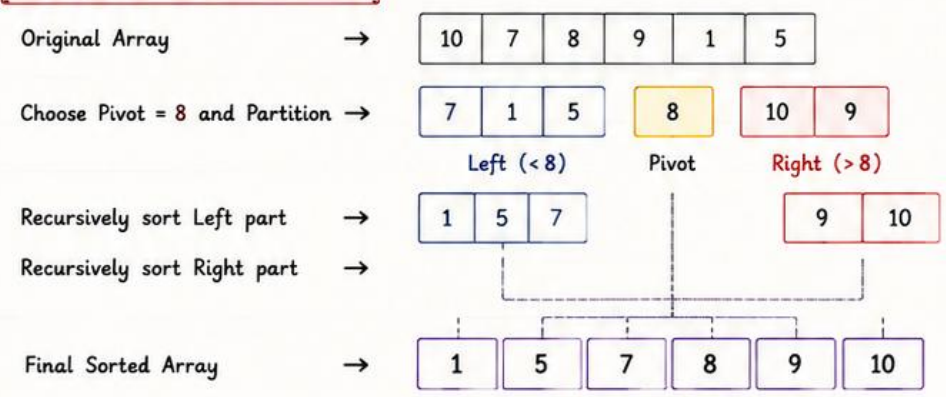
→

2. CONQUER
Recursively apply Quick Sort on the Left part and Right part

→

3. COMBINE
No explicit combine step is needed.
The array is sorted in-place during partitioning.

VISUAL REPRESENTATION



Key Insight

Quick Sort doesn't need extra space to merge like Merge Sort.
It sorts in-place by cleverly partitioning the array.

RECURRENCE RELATION

Let T(n) be the time taken to sort an array of size n.

$$T(n) = T(k) + T(n - k - 1) + \Theta(n)$$

Where:

- k = number of elements on one side of pivot
- (n - k - 1) = number of elements on the other side
- Θ(n) = time for partitioning

WHY THIS APPROACH?

- ✓ Reduces the problem size at each step.
- ✓ Solves subproblems of the same type.
- ✓ No extra merge step (in-place sorting).
- ✓ Works efficiently in practice (especially with good pivot choice).

Divide the problem → Conquer the subproblems → Get the sorted solution!

Date : __/__/__

Divide
Conquer
Combine

3. PARTITIONING + PIVOT SELECTION

Partitioning is the process of rearranging the array such that all elements smaller than pivot are on the left, and all elements greater than pivot are on the right.

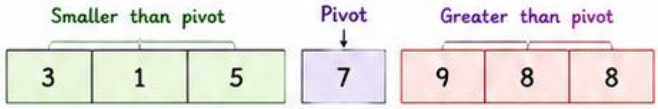
WHAT IS PARTITIONING?

- Choose a pivot element.
- Rearrange the array so that:
  - Left side: elements < pivot
  - Right side: elements > pivot
- Pivot is placed at its correct sorted position.
- Return the pivot index (final position).

EXAMPLE

Array: [9, 3, 7, 1, 8, 2, 5]

Choose pivot = 7



Pivot index = 4 (0-based indexing)

PIVOT SELECTION METHODS

The choice of pivot greatly affects performance.

(1) First Element

Choose the first element as pivot.

Simple but can give worst case for sorted arrays.

Example:

9	3	7	1	8

Pivot = 9

(2) Last Element

Choose the last element as pivot.

Better than first in some cases.

Example:

9	3	7	1	8

Pivot = 8

(3) Random Element

Choose a random element as pivot.

Reduces chances of worst case.

Example:

9	3	7	1	8

Pivot = 7 (random)

(4) Median of Three

Choose median of first, middle and last elements as pivot.

Best balance in most cases.

Example:

9	3	7	1	8

first, middle, last

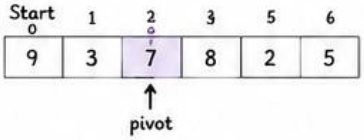
Values = {9, 7, 8} → Median = 8

Pivot = 8

PARTITIONING PROCESS (Using pivot = 7)

Example Array: [9, 3, 7, 1, 8, 2, 5]

Step 1: Start



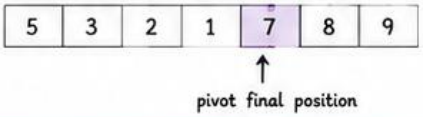
Step 2: i moves right, j moves left



Step 3: Swap 9 and 5



Step 4: Continue until i > j, then place pivot



Partition Result

All elements left of 7 are smaller.

All elements right of 7 are greater.

Pivot is in its correct position.

Index = 4

THINGS TO REMEMBER

- ✓ Partitioning does not sort the subarrays completely.
- ✓ It only puts the pivot in the correct position.
- ✓ After partitioning, recursively apply Quick Sort on left and right subarrays.
- ✓ Good pivot → Balanced partitions → Better performance.
- ✓ Bad pivot → Unbalanced partitions → Worst performance.

IMPACT OF PIVOT CHOICE

Pivot Choice	Best Case	Average Case	Worst Case
Good (Balanced)	O(n log n)	O(n log n)	O(n log n)
Average	O(n log n)	O(n log n)	$O(n^2)$
Bad (Highly Unbalanced)	O(n log n)	$O(n^2)$	$O(n^2)$

KEY TAKEAWAY

Partitioning is the heart of Quick Sort.
Choosing a good pivot leads to balanced partitions and optimal performance!

Good Partition + Good Pivot = Fast Quick Sort!

Divide
Conquer
Combine

4. LOMUTO PARTITION SCHEME

Date : __/__/__

Lomuto Partition Scheme is a simple and widely used method for partitioning. It chooses the last element as pivot and places it in its correct sorted position after partitioning.

IDEA

- Choose the last element of the array as pivot.
- Maintain an index i for the smaller element.
- Traverse the array from left to right.
  - If arr[j] ≤ pivot → increment i and swap arr[i] and arr[j].
  - If arr[j] > pivot → do nothing.
- Finally, swap pivot (arr[high]) with arr[i+1].

ALGORITHM

```
LomutoPartition(arr, low, high)
1. pivot = arr[high]          // pivot is last element
2. i = low - 1                // index of smaller element
3. for j = low to high - 1
4.     if arr[j] <= pivot
5.         i = i + 1
6.         swap arr[i] and arr[j]
7. swap arr[i + 1] and arr[high]   // put pivot in correct place
8. return i + 1                    // pivot index
```

VISUAL EXAMPLE (Pivot = last element)

Array: [9, 3, 7, 1, 8, 2, 5] low = 0, high = 6, pivot = 5

	0	1	2	3	4	5	6
Initial	9	3	7	1	8	2	5
	i = -1						
j = 0: arr[0] = 9 > 5 No swap	9	3	7	1	8	2	5
	i = -1						
j = 1: arr[1] = 3 ≤ 5 Swap	3	9	7	1	8	2	5
		i = 0					
j = 2: arr[2] = 7 > 5 No swap	3	9	7	1	8	2	5
		i = 0					
j = 3: arr[3] = 1 ≤ 5 Swap	3	1	7	9	8	2	5
			i = 1				
j = 4: arr[4] = 8 > 5 No swap	3	1	7	9	8	2	5
			i = 1				
j = 5: arr[5] = 2 ≤ 5 Swap	3	1	7	9	8	2	5
			i = 2				

Final: swap arr[i+1] and arr[high] (swap arr[3] and arr[6])

	0	1	2	3	4	5	6
	3	1	2	5	8	9	7
				pivot index = 3			

C IMPLEMENTATION

```c
int lomutoPartition(int arr[], int low, int high) {
    int pivot = arr[high];    // pivot is last element
    int i = low - 1;          // index of smaller element

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            // swap arr[i] and arr[j]
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    // swap arr[i+1] and arr[high] (put pivot in place)
    int temp = arr[i+1];
    arr[i+1] = arr[high];
    arr[high] = temp;

    return i + 1;    // pivot index
}
```

DRY RUN SUMMARY

→ Pivot chosen = 5 (last element)
→ Elements ≤ 5 are moved to the left
→ Elements > 5 remain on the right
→ Pivot placed at index 3
- Left part: [3, 1, 2]
→ Right part: [8, 9, 7]

COMPLEXITY

- Time Complexity:
  - Best Case: O(n log n)
  - Average Case: O(n log n)
  - Worst Case: O(n²)
- Space Complexity:
  - O(1) extra space (in-place)

PROS & CONS

Pros
- Simple to implement
- In-place partitioning
- Works well on average

Cons
- Worst case O(n²) if array is already sorted or reverse sorted
- Pivot choice (last element) can be bad in some cases

KEY TAKEAWAY

Lomuto scheme is easy to understand and implement.
Its performance depends heavily on the choice of pivot.

Date : __/__/__

Divide
Conquer
Combine

5. HOARE PARTITION SCHEME

Hoare Partition Scheme is another efficient method for partitioning. It chooses any element as pivot (commonly the first element) and uses two pointers that move towards each other from both ends.

CONCEPT

- Choose the first element as pivot (commonly).
- Use two pointers:
  - i starts from left (low - 1) and moves right until it finds an element ≥ pivot.
  - j starts from right (high + 1) and moves left until it finds an element ≤ pivot.
- If i < j, swap arr[i] and arr[j].
- When i ≥ j, return j (partition index).
  All elements at index ≤ j are ≤ pivot and all elements at index > j are ≥ pivot.

ALGORITHM

HoarePartition(arr, low, high)

1. pivot = arr[low] // choose first element as pivot
2. i = low - 1
3. j = high + 1
4. Repeat forever
   a) Move i right, while arr[i] < pivot
   b) Move j left, while arr[j] > pivot
   c) If i ≥ j, return j
   d) Swap arr[i] and arr[j]

WORKING (Pivot = first element)

Array : [10, 7, 8, 9, 1, 5] low = 0, high = 5, pivot = 10

Step	i	j	Action	Array State
Initial	-1	6	i = low-1, j = high+1	[10, 7, 8, 9, 1, 5] (pivot)
1	0	5	i → arr[0] = 10 (>=10) stop j → arr[5] = 5 (<=10) stop	[10, 7, 8, 9, 1, 5]
2	0	5	i < j → swap arr[0], arr[5]	[5, 7, 8, 9, 1, 10]
3	1	4	i → arr[1] = 7 (<10) → i++ stop at i = 1 j → arr[4] = 1 (<=10) → j-- stop at j = 4	[5, 7, 8, 9, 1, 10]
4	1	4	i < j → swap arr[1], arr[4]	[5, 1, 8, 9, 7, 10]
5	2	3	i → arr[2] = 8 (<10) → i++ stop at i = 2 j → arr[3] = 9 (<=10) → j-- stop at j = 3	[5, 1, 8, 9, 7, 10]
6	2	3	i < j → swap arr[2], arr[3]	[5, 1, 9, 8, 7, 10]
7	3	2	i = 3, j = 2 → i ≥ j return j = 2	[5, 1, 9, 8, 7, 10]

Partition index = 2
Elements at index ≤ 2 are ≤ pivot, and index > 2 are ≥ pivot.

C IMPLEMENTATION

```c
int hoarePartition(int arr[], int low, int high) {
  int pivot = arr[low];    // first element as pivot
  int i = low - 1;
  int j = high + 1;
  while (1) {
    do {
        i++;                // move i right
    } while (arr[i] < pivot);

    do {
        j--;                // move j left
    } while (arr[j] > pivot);

    if (i >= j)
        return j;           // partition index

    // swap arr[i] and arr[j]
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
  }
}
```

RETURN VALUE

Returns index j such that:

- Left part (0 to j) → elements ≤ pivot
- Right part (j+1 to n-1) → elements ≥ pivot

COMPARISON WITH LOMUTO

	Hoare	Lomuto
Pivot	Usually first element	Always last element
Pointers	Two pointers (i and j)	One pointer (i)
Swaps	Fewer swaps	More swaps
Return	Returns index j	Returns index i+1
Performance	Better in practice	Slightly slower

COMPLEXITY

- Time Complexity:
  - Best Case : O(n log n)
  - Average Case : O(n log n)
  - Worst Case : $O(n^2)$
- Space Complexity:
  - O(log n) average
  - O(n) worst case (due to recursion stack)

ADVANTAGES

- ✓ Faster than Lomuto in practice (less swaps).
- ✓ Works well with many duplicate elements.
- ✓ Used in many standard library implementations.

DISADVANTAGES

- ✗ Slightly harder to understand.
- ✗ Return index is j (not i+1 like Lomuto).
- ✗ Worst case $O(n^2)$ if pivot choice is poor.

KEY TAKEAWAY

Hoare Partition Scheme is pointer-efficient and swap-efficient.
Good pivot choice is key to achieving best performance!

Date : __/__/__

Divide
Conquer
Combine

6. QUICK SORT ALGORITHM + DRY RUN

Quick Sort is a Divide and Conquer algorithm.
It partitions the array around a pivot and recursively sorts the left and right subarrays.

QUICK SORT ALGORITHM

```
QuickSort(arr, low, high)
1. if low < high
   a) pi = Partition(arr, low, high)   // pi = pivot index
   b) QuickSort(arr, low, pi - 1)      // sort left part
   c) QuickSort(arr, pi + 1, high)     // sort right part
2. return
```

PARTITION (Using Lomuto Scheme)

```
Partition(arr, low, high)
1. pivot = arr[high]          // last element as pivot
2. i = low - 1                // index of smaller element
3. for j = low to high - 1
     a) if arr[j] <= pivot
          i = i + 1
          swap arr[i] and arr[j]
4. swap arr[i + 1] and arr[high]   // put pivot in place
5. return i + 1               // pivot index
```

DRY RUN EXAMPLE

Array: [10, 7, 8, 9, 1, 5] low = 0, high = 5

Using Lomuto Partition (pivot = last element)

Step 1: Partition with pivot = 5

Initial Array

10	7	8	9	1	5
0	1	2	3	4	5

pivot = 5, i = -1

j = 0: arr[0] = 10 > 5 (no swap)	10	7	8	9	1	5	i = -1
j = 1: arr[1] = 7 > 5 (no swap)	10	7	8	9	1	5	i = -1
j = 2: arr[2] = 8 > 5 (no swap)	10	7	8	9	1	5	i = -1
j = 3: arr[3] = 9 > 5 (no swap)	10	7	8	9	1	5	i = -1
j = 4: arr[4] = 1 <= 5 (swap)	10	7	8	9	1	5	i = 0

Place pivot at correct position (swap arr[i+1] and arr[high])

1	7	8	9	10	5	pi = i + 1 = 1

Pivot 5 is placed at index 1.

Left part (low to pi-1) → indices [0..0]

Right part (pi+1 to high) → indices [2..5]

Step 2: Sort Left Part [0..0]

Array: [1] → Already sorted

Step 3: Sort Right Part [2..5] → [8, 9, 10, 5]

Partition with pivot = 5

8	9	10	5
2	3	4	5

Left part [2..1] → empty

After partition

5	9	10	8	pi = 2

Right part [3..5] → [9, 10, 8]

Step 4: Sort Right Part [3..5] → [9, 10, 8]

Partition with pivot = 8

9	10	8
3	4	5

Left part [3..2] → empty

After partition

8	10	9	pi = 3

Right part [4..5] → [10, 9]

Step 5: Sort Right Most Part [4..5] → [10, 9]

Partition with pivot = 9

10	9
4	5

Left part [4..3] → empty

After partition

9	10	pi = 4

Right part [5..5] → empty

FINAL SORTED ARRAY

1	5	7	8	9	10
0	1	2	3	4	5

Good Pivot Choice
Better Performance!

KEY TAKEAWAY

- ✓ Choose a pivot and partition the array.
- ✓ Recursively apply Quick Sort on left and right parts.
- ✓ Base case: when low >= high, return.
- ✓ Average Time Complexity: $O(n \log n)$
- ✓ Worst Case Time Complexity: $O(n^2)$

Divide
Conquer
Combine

7. RECURSIVE IMPLEMENTATION IN C, C++ & PYTHON

Date : __/__/__

Quick Sort is a recursive algorithm.
It partitions the array and recursively sorts the left and right subarrays.

Algorithm Summary

Choose Pivot → Partition Array → Recur for Left Part → Recur for Right Part → Array Sorted

C IMPLEMENTATION

```c
#include <stdio.h>

// Lomuto Partition
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

// Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);  // Left
        quickSort(arr, pi + 1, high); // Right
    }
}
// Driver Code
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    printArray(arr, n);
    return 0;
}
```

Explanation

- partition() places pivot in its correct position and returns its index.
- quickSort() recursively sorts left and right subarrays.
- Base case: when low >= high (0 or 1 element)

C++ IMPLEMENTATION

```cpp
#include <bits/stdc++.h>
using namespace std;

// Lomuto Partition
int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

// Quick Sort
void quickSort(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Driver Code
int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    int n = arr.size();
    quickSort(arr, 0, n - 1);

    for (int x : arr) cout << x << " ";
    cout << endl;
    return 0;
}
```

Explanation

- Uses STL vector and swap().
- Logic same as C implementation.
- less code for printing using range-based loop.

PYTHON IMPLEMENTATION

```python
# Lomuto Partition
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

# Quick Sort
def quickSort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)

        quickSort(arr, low, pi - 1)   # Left
        quickSort(arr, pi + 1, high)  # Right

# Driver Code
if __name__ == "__main__":
    arr = [10, 7, 8, 9, 1, 5]
    n = len(arr)
    quickSort(arr, 0, n - 1)
    print(" ".join(map(str, arr)))
```

Explanation

- Python uses list and tuple unpacking for swap.
- Clean and readable syntax.
- Recursion same as C/C++.

Sample Run

Input Array : [10, 7, 8, 9, 1, 5]
Sorted Array : [1, 5, 7, 8, 9, 10]

Complexity

Best Case : O(n log n)
Average Case : O(n log n)
Worst Case : $O(n^2)$
Space Complexity: O(log n) (recursion stack)

Notes

- ✓ Choose a good pivot for better performance.
- ✓ Worst case occurs when array is already sorted or reverse sorted and pivot is always first or last element.

Key Takeaway

Three languages, same logic!
Understand recursion + partitioning well and you can implement anywhere.

Divide
Conquer
Combine

8. TIME & SPACE COMPLEXITY + BEST, AVERAGE & WORST CASES

Date : __/__/__

Quick Sort is an efficient Divide & Conquer algorithm.
Its performance depends mainly on the choice of pivot.

1. BASIC IDEA

- Quick Sort partitions the array around a pivot.
- Elements smaller than pivot go to the left.
- Elements greater than pivot go to the right.
- Recursively sort the left and right subarrays.

Array
[10, 7, 8, 9, 1, 5]
pivot = 5

< pivot: 1
> pivot: 10 7 8 9

2. TIME COMPLEXITY OVERVIEW

Case	Condition	Time Complexity	Recurrence Relation
Best Case	Pivot always divides array into two equal parts	$\Theta(n \log n)$	$T(n) = 2T(n/2) + \Theta(n)$
Average Case	Pivot divides array into unbalanced parts on average	$\Theta(n \log n)$	$T(n) = 2T(n/2) + \Theta(n)$
Worst Case	Pivot is always the smallest or largest element	$\Theta(n^2)$	$T(n) = T(n-1) + \Theta(n)$

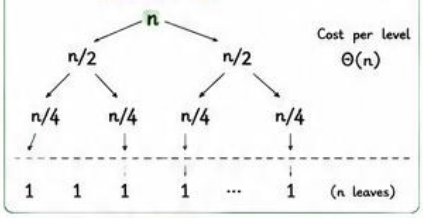
3. BEST, AVERAGE & WORST CASE ANALYSIS

BEST CASE

- Pivot divides the array into two equal halves at every step.
- Recurrence:
  $T(n) = 2T(n/2) + \Theta(n)$
- By Master Theorem:
  $a = 2,\ b = 2,\ f(n) = \Theta(n)$
  $n^{\log_b a} = n^{\log_2 2} = n^1$
  $f(n) = \Theta(n^1) \Rightarrow$ Case 2

$T(n) = \Theta(n \log n)$

Recursion Tree



Height = $\log_2 n$
Total Cost = $\Theta(n \log n)$

Example
Pivot is the middle element each time.
Array: [4, 2, 6, 1, 3, 5, 7]
Balanced partitions.

AVERAGE CASE

- Pivot divides the array into unbalanced subarrays on average.
- Recurrence (same as best case):
  $T(n) = 2T(n/2) + \Theta(n)$
- By Master Theorem:

$T(n) = \Theta(n \log n)$

Explanation
Most of the time, partitions are reasonably balanced, leading to n levels of recursion and $\Theta(n)$ work per level.

Example
Random pivot selection.
Array: [10, 7, 8, 9, 1, 5]
On average, we get balanced splits.

WORST CASE

- Pivot is always the smallest or largest element.
- One subarray has $(n-1)$ elements and the other has 0.
- Recurrence:
  $T(n) = T(n-1) + \Theta(n)$
- Unfolding:

$$T(n) = T(n-1) + \Theta(n)$$
$$= T(n-2) + \Theta(n-1) + \Theta(n)$$
$$= \cdots$$
$$= T(1) + \Theta(n + (n-1) + \cdots + 2)$$
$$= \Theta(n^2)$$

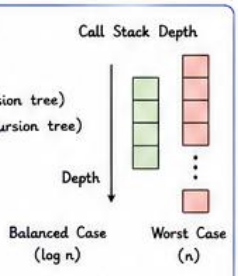
$T(n) = \Theta(n^2)$

Example
Pivot is always first or last element.
Sorted array: [1, 2, 3, 4, 5, 6]
Reverse sorted array: [6, 5, 4, 3, 2, 1]

4. SPACE COMPLEXITY

- In-place sorting algorithm (no extra array used).
- Extra space is used only by recursion stack.
- Space Complexity = O(log n) on average (balanced recursion tree)
- Space Complexity = O(n) in worst case (unbalanced recursion tree)

Case	Space Complexity
Best Case	O(log n)
Average Case	O(log n)
Worst Case	O(n)



5. COMPLEXITY SUMMARY TABLE

Case	Partition Quality	Time Complexity	Space Complexity
Best	Two equal halves	$\Theta(n \log n)$	$O(\log n)$
Average	Unequal but balanced on average	$\Theta(n \log n)$	$O(\log n)$
Worst	One side has $(n-1)$ elements, other has 0	$\Theta(n^2)$	$O(n)$

6. KEY POINTS

- ☑ Good pivot choice → balanced partitions → $\Theta(n \log n)$
- ☑ Poor pivot choice → unbalanced partitions → $\Theta(n^2)$
- ☑ Randomized or median-of-three pivot helps avoid worst case.
- ☑ Quick Sort is usually faster in practice due to low constant factors and good cache performance.

7. KEY TAKEAWAY

- ★ Best Case: $\Theta(n \log n)$ time, $O(\log n)$ space
- ★ Average Case: $\Theta(n \log n)$ time, $O(\log n)$ space
- ★ Worst Case: $\Theta(n^2)$ time, $O(n)$ space

Good Pivot = Fast Quick Sort!

Divide
Conquer
Combine

9. QUICK SORT OPTIMIZATIONS + COMMON PROBLEMS

Date : __/__/__

Quick Sort is already efficient (O(n log n) on average).
With smart optimizations, we can make it faster in practice and avoid worst cases.

PART A: OPTIMIZATIONS

1. Better Pivot Selection

Choosing a good pivot avoids highly unbalanced partitions.

- First element (simple but risky) ✗
- Last element (same risk) ✗
- Random element (good) ✓
- Median of Three (low, mid, high) (best in practice) ✓

Median of Three Example

Array: [12, 7, 9, 3, 15, 8, 10]
low = 0 (12), mid = 3 (3),
high = 6 (10)
Median of {12, 3, 10} → 10
Use 10 as pivot.

2. Randomized Quick Sort

Pick a random index as pivot.

- Avoids worst case for already sorted or reverse sorted arrays.
- Expected time remains O(n log n).

Example

Array:

10	20	30	40	50	60
0	1	2	3	4	5

Random index = 2 → pivot = 30

3. Cut-off to Insertion Sort (Hybrid Quick Sort)

For small subarrays (size ≤ k, typically 10 to 20), use Insertion Sort instead of Quick Sort.

- Reduces overhead of recursion.
- Improves performance in practice.

Example
Subarray: [5, 2, 4, 1, 3]
Size = 5 ≤ 10 →
Use Insertion Sort

4. Tail Recursion Elimination

Always recurse for smaller part first and use leop for the larger part.

- Reduces maximum recursion depth.
- Saves stack space.

Idea
If left part < right part
- Recur for left
- Loop for right

Else
- Recur for right
- Loop for left

5. 3-Way Partitioning (Dutch National Flag)

Useful when array has many duplicate elements.
Partitions into three parts:
< pivot = pivot > pivot

- Greatly reduces comparisons.
- Handles duplicates efficiently.

Example
Array: [4, 3, 4, 4, 2, 4, 1, 4]

<4			=4				>4
3	2	1	4	4	4	4	4	4

6. Avoid Unnecessary Swaps

Swap only when elements are actually on wrong sides.
Reduces constant factors → faster.

Bad Swap
swap(5, 5) ✗

Good Approach
if (arr[i] != arr[j]) swap ✓

7. Use Iterative Quick Sort (Optional)

Use explicit stack instead of recursion.

- Avoids stack overflow for large inputs.
- More control on memory.

Idea
- Push larger part indices
- Process smaller part first

OPTIMIZED QUICK SORT = SMART PIVOT + SMALL CUT-OFF + LESS RECURSION + HANDLE DUPLICATES = FASTER & MORE ROBUST ;

PART B: COMMON PROBLEMS

1. Worst Case Occurs

Problem: When array is already sorted or reverse sorted and pivot is always first or last element.

Example:
Sorted Array: [1, 2, 3, 4, 5, 6, 7]
Pivot = first element (1)
→ Each partition gives (0, n-1), (0, n-2), ...
→ Time Complexity = $O(n^2)$

Solution:
Use Randomized or Median-of-Three pivot.
Always avoid fixed pivot (first/last).

2. Stack Overflow (Large Input)

Problem: Deep recursion in worst case.
Example: n = 10^6 sorted array.
→ Recursion depth = n
→ Stack Overflow Error.

Solution:
- Use Tail Recursion Elimination.
- Or use Iterative Quick Sort with explicit stack.

3. Many Duplicate Elements

Problem: Standard 2-way partition is inefficient.
Example:
Array: [4, 4, 4, 4, 4, 4, 4,]
→ Many unnecessary swaps and unbalanced partitions.

Solution:
Use 3-Way Partitioning (Dutch National Flag).
Handles duplicates in O(n log n) time.

4. Unnecessary Swaps

Problem: Swapping elements even when they are already on correct side.
Example:
swap(arr[i], arr[j]) even when arr[i] == arr[j]

Solution:
Check before swap.
if (arr[i] != arr[j]) swap(arr[i], arr[j]);

5. Inefficient on Small Subarrays

Problem: Recursive calls on very small subarrays cause extra overhead.

Solution:
Use Insertion Sort for subarray size ≤ 10 (or 16).

COMPLEXITY REMINDER

Case	Time Complexity	Space Complexity
Best Case (balanced)	O(n log n)	O(log n)
Average Case	O(n log n)	O(log n)
Worst Case (unbalanced)	$O(n^2)$	O(n)

KEY TAKEAWAY

- ☑ Good pivot → Balanced partitions → O(n log n)
- ☑ Handle duplicates → Use 3-way partition
- ☑ Small subarrays → Use Insertion Sort
- ☑ Reduce recursion → Tail elimination / Iterative
- ☑ Avoid unnecessary swaps → Faster constant factors



10. QUICK SORT
INTERVIEW QUESTIONS + CODING CHEAT SHEET



Quick Sort = Divide and Conquer. Choose a pivot, partition the array, recursively sort left and right parts.

A. INTERVIEW QUESTIONS (with answers in short)

1. What is Quick Sort?
It is a Divide and Conquer sorting algorithm. It picks a pivot, partitions the array around it and recursively sorts the left and right parts.

2. What is the time complexity of Quick Sort?
Best Case : O(n log n)
Average Case : O(n log n)
Worst Case : O(n²)

3. What is the space complexity?
O(log n) on average due to recursion stack.
O(n) in worst case (skewed recursion).

4. When does Quick Sort give worst case?
- When pivot is always the smallest or largest element.
- Example: already sorted or reverse sorted array with fixed pivot (first/last element).

5. How to avoid worst case?
- Use Randomized Pivot or Median-of-Three Pivot.
- Always avoid fixed pivot (first/last).
- Use 3-way partition for duplicates.

6. Is Quick Sort stable?
- No, Quick Sort is not stable. (Relative order of equal elements may change).

7. Is Quick Sort in-place?
- Yes, it is an in-place sorting algorithm. It requires only O(log n) extra space on average (recursion stack).

8. What is the best case for Quick Sort?
When partitions are always perfectly balanced. (Pivot is always the median element)
Time Complexity = O(n log n)

9. How does partitioning work?
It rearranges elements so that:
- All elements ≤ pivot are on the left,
- All elements > pivot are on the right.
- Pivot ends up in its correct position.

10. What is the difference between Lomuto and Hoare partition schemes?

Feature	Lomuto	Hoare
Pivot Position	Last element	First element
Return	Index of pivot	Index j (split point)
Swaps	More swaps	Fewer swaps
Implementation	Simple	Slightly complex

11. How to handle duplicate elements efficiently?
Use 3-Way Partitioning (Dutch National Flag) which divides into < pivot, = pivot, > pivot. Avoids unnecessary swaps and improves performance.

12. Quick Sort vs Merge Sort?

Feature	Quick Sort	Merge Sort
Time (avg)	O(n log n)	O(n log n)
Time (worst)	O(n²)	O(n log n)
Space	O(n²)	O(n)
In-place	Yes	No
Stable	No	Yes

Quick Sort is faster in practice due to less extra space and cache friendliness.

B. CODING CHEAT SHEET

1. Lomuto Partition (Pivot = last element)

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;            // index of smaller element
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;               // pivot index
}
```

2. Hoare Partition (Pivot = first element)

```
int hoarePartition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low - 1;
    int j = high + 1;
    while (1) {
        do { i++; } while (arr[i] < pivot);
        do { j--; } while (arr[j] > pivot);
        if (i >= j) return j;     // split point
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

3. Quick Sort (Using Lomuto Partition)

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);    // left
        quickSort(arr, pi + 1, high);   // right
    }
}
```

4. Driver Code (C/C++)

```
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output
1 5 7 8 9 10

5. Python Implementation (Lomuto)

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i+1], arr[high] = arr[high], arr[i+1
    return i + 1
```

```
def quickSort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quickSort(arr, low, pi - 1)
        quickSort(arr, pi + 1, high)

# Driver
arr = [10, 7, 8, 9, 1, 5]
quickSort(arr, 0, len(arr) - 1)
print(arr)
```

[1, 5, 7, 8, 9, 10]

COMPLEXITY SUMMARY

Case	Time Complexity	Space Complexity
Best Case	O(n log n)	O(log n)
Average Case	O(n log n)	O(log n)
Worst Case	O(n²)	O(n)

KEY OPTIMIZATIONS

- ✓ Use Randomized or Median-of-Three pivot.
- ✓ Use 3-Way Partitioning for duplicates.
- ✓ Use Insertion Sort for small subarrays (≤ 10).
- ✓ Eliminate tail recursion / use iterative version.
- ✓ Avoid unnecessary swaps.

QUICK REMINDER

Choose Pivot → Partition → Recur Left → Recur Right → Array Sorted





Good Pivot + Good Partition + Smart Optimizations = Fast & Robust Quick Sort!